

扩展欧几里得算法

求 $ax + by = \gcd(a, b)$ 的一组解，函数返回值是 $\gcd(a, b)$ ， x, y 在迭代中途求解

该函数也可用于求逆元，因为 a, b 互素的时候求解 $ax + by = 1$ ，即 $ax \equiv 1 \pmod{b}$ ，但 x 有可能是负数

```
typedef long long ll;
ll Exgcd(ll a, ll b, ll& x, ll& y) {
    if (!b) {
        x = 1, y = 0;
        return a;
    }
    ll d = Exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

扩欧求解线性同余方程 $ax \equiv c \pmod{b}$ ，转化成 $ax + by = c$ 求解，函数返回是否有解， x, y 是 $ax + by = c$ 的一组解，通解 $x = x_0 + bt; y = y_0 - at$ (t 取任意整数)

```
bool liEu(int a, int b, int c, int& x, int& y) {
    int d = Exgcd(a, b, x, y);
    if (c % d != 0) return false;
    int k = c / d;
    x *= k;
    y *= k;
    return true;
}
```

中国剩余定理

求解线性同余方程组 $x \equiv a_i \pmod{r_i}$

```

typedef long long ll; //很可能long long也会爆，要开__int128
ll CRT(vector<ll> &r, vector<ll> &a) {
    ll n = 1, ans = 0, k = a.size();
    for (auto x : r) n = n * x;
    for (int i = 0; i < k; i++) {
        ll m = n / r[i], R, y;
        Exgcd(m, r[i], R, y); // 见扩欧模版
        ans = (ans + a[i] * m * R % n) % n;
    }
    return (ans % n + n) % n;
}

```

扩展中国剩余定理

处理模数不互质的情况

```

ll ExCRT(vector<ll>&r,vector<ll>&a) {
    ll R = r[1], A = a[1];
    auto mix = [&](ll aa, ll rr) {
        ll x, y;
        liEu(R, rr, aa - A, x, y);
        A += R * x;
        R = lcm(R, rr);
        A %= R;
        if (A < 0) A += R;
    };
    for (int i = 2; i < r.size(); i++)
        mix(a[i], r[i]);
    ll ans = A % R;
    if (ans < 0) ans += R;
    return ans;
}

```

Min_25筛

用途：定义积性函数 $f(x)$ ，求解 $\sum_{i=1}^n f(i)$

使用说明：要将 $f(x)$ 多项式拆成单项式，然后分别计算递推函数g

下板题意为：定义积性函数 $f(x)$ ，且 $f(p^k) = p^k(p^k - 1)$ (p 是一个质数)

```

void min_25(int n) {
    const int MOD = 1000000007, inv2 = 500000004, inv3 = 333333336; //inv2、inv3分别表示2和
    int sq = sqrt(n);
    vector<int>g1(sq * 3), g2(sq * 3); //递推函数,数量依据多项式拆出来的单项式个数
    vector<int>sp1(sq * 3), sp2(sq * 3); //前i项质数的多项式前缀和
    vector<int>w(sq * 3); //离散化存储要处理的数
    vector<int>idx1(sq * 3), idx2(sq * 3); //记录离散化下标

    auto getPrime = [&](int n){
        vector<bool> vis(n + 1);
        vector<int> prime;
        prime.push_back(1);
        vis[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (!vis[i]) {
                prime.push_back(i);
                sp1[prime.size() - 1] = (sp1[prime.size() - 2] +
                sp2[prime.size() - 1] = (sp2[prime.size() - 2] +
                //求素数的函数和
            }
            for (auto p : prime) {
                if (p > 1) {
                    if (i * p > n) { break; }
                    vis[i * p] = 1;
                    if (i % p == 0) { break; }
                }
            }
        }

        return prime;
    };

    vector<int> p = getPrime(sq);
    function<int(int, int)> S = [&](int x, int y) ->int {
        if (p[y] >= x) return 0;
        int k = (x <= sq ? idx1[x] : idx2[n / x]);
        int ans = (g2[k] - g1[k] + MOD - (sp2[y] - sp1[y]) + MOD) % MOD; //质数贡献
        for (int i = y + 1; i < p.size() && p[i] * p[i] <= x; i++) //合数贡献
        {
            int pe = p[i];
            for (int e = 1; pe <= x; e++, pe = pe * p[i])
            {
                int xx = pe % MOD;

```

```

        ans = (ans + xx * (xx - 1) % MOD * (S(x / pe, i) + (e != 1
    }
}
return ans % MOD;
};
int d = 0, cnt = 0;
while (d < n) {
    d++;
    w[++cnt] = n / d;
    int tmp = w[cnt] % MOD;

    //全部初始化为g(i,0)，即函数前缀和(包括素数和合数)
    g2[cnt] = tmp * (tmp + 1) / 2 % MOD * (2 * tmp + 1) % MOD * inv3 % MOD - 1; //
    g1[cnt] = tmp * (tmp + 1) / 2 % MOD - 1;

    //数论分块
    if (n / d <= sq) idx1[n / d] = cnt;
    else idx2[n / (n / d)] = cnt;
    d = n / (n / d);
}

for (int i = 1; i < p.size(); i++)
    for (int j = 1; j <= cnt && p[i] * p[i] <= w[j]; j++)
    {
        int k = (w[j] / p[i] <= sq ? idx1[w[j] / p[i]] : idx2[n / (w[j] / p
        g1[j] -= p[i] * (g1[k] - sp1[i - 1] + MOD) % MOD;
        g2[j] -= p[i] * p[i] % MOD * (g2[k] - sp2[i - 1] + MOD) % MOD;
        g1[j] %= MOD, g2[j] %= MOD;
        if (g1[j] < 0) g1[j] += MOD;
        if (g2[j] < 0) g2[j] += MOD;
    }
cout << (S(n, 0) + 1) % MOD;
}

```

也可以用于求非积性函数的质数项前缀和，只需最后计算S的时候去掉合数的贡献即可，例如：Min_25
筛求区间素数个数(即n以内素数个数)

```

int min_25(int n) {
    int sq = sqrt(n);
    vector<int>g1(sq * 3);
    vector<int>sp1(sq * 3);
    vector<int>w(sq * 3);
    vector<int>idx1(sq * 3), idx2(sq * 3);
    auto getPrime = [&](int n) {
        vector<bool> vis(n + 1);
        vector<int> prime;
        prime.push_back(1);
        vis[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (!vis[i]) {
                prime.push_back(i);
                sp1[prime.size() - 1] = (sp1[prime.size() - 2] + 1);
            }
            for (auto p : prime) {
                if (p > 1) {
                    if (i * p > n) { break; }
                    vis[i * p] = 1;
                    if (i % p == 0) { break; }
                }
            }
        }
        return prime;
    };
    vector<int> p = getPrime(sq);
    function<int(int, int)> S = [&](int x, int y) ->int {
        if (p[y] >= x) return 0;
        int k = (x <= sq ? idx1[x] : idx2[n / x]);
        int ans = g1[k] - sp1[y];
        return ans;
    };
    int d = 0, cnt = 0;
    while (d < n) {
        d++;
        w[++cnt] = n / d;
        int tmp = w[cnt];
        g1[cnt] = tmp - 1;
        if (n / d <= sq) idx1[n / d] = cnt;
        else idx2[n / (n / d)] = cnt;
        d = n / (n / d);
    }
}

```

```

    for (int i = 1; i < p.size(); i++)
        for (int j = 1; j <= cnt && p[i] * p[i] <= w[j]; j++)
        {
            int k = (w[j] / p[i] <= sq ? idx1[w[j] / p[i]] : idx2[n / (w[j] / p
            g1[j] -= g1[k] - sp1[i - 1];
        }
    return S(n, 0);
}

```

Lucas定理

在模数较小的情况下快速计算排列组合数

```

typedef long long ll;
ll Lucas(ll n, ll k, ll p) {
    if (k == 0) return 1;
    return (C(n % p, k % p, p) * Lucas(n / p, k / p, p)) % p;
}

```

拓展欧拉定理

欧拉函数定义

欧拉函数，符号记为 φ 。 $\varphi(n)$ 表示有多少个正整数 i ，满足 $\gcd(i, n) = 1$ 且 $i \leq n$ 。用数学符号表述为：

$$\varphi(n) = \sum_{i=1}^n [\gcd(i, n) = 1]$$

特殊性质

若 n 为质数，则 $\varphi(n) = n - 1$ 。根据质数的定义得证。

若 $n = p^k$ ，其中 p 为质数，则 $\varphi(n) = p^k - p^{k-1} = (p - 1)p^{k-1}$ 。因为 n 只有 p 一个素因子，所以除了 p 的倍数之外其余数都与 n 互质。

对于任意 n ，由唯一分解定理 $n = \prod_{i=1}^k p_i^{a_i}$ ，有：

$$\varphi(n) = n \times \prod_{i=1}^k (1 - 1/p_i)$$

注意同一个质因子只计算一次。

若 $\gcd(n, m) = 1$ ，则 $\varphi(nm) = \varphi(n)\varphi(m)$ （即 $\varphi(x)$ 为积性函数）。

证明：根据唯一分解定理，设

$$n = p_1^{a_1} p_2^{a_2} \dots p_s^{a_s}, \quad m = q_1^{b_1} q_2^{b_2} \dots q_t^{b_t}$$

因为 $\gcd(n, m) = 1$ ，所以不存在 $p_i = q_j$ 。由性质 3 可得 $\varphi(nm) = \varphi(n)\varphi(m)$ 。

欧拉函数计算

线性筛可以处理各种积性函数的计算，欧拉函数也不例外。我们可以在 $O(n)$ 时间内计算出所有 $\varphi(i)$ ($1 \leq i \leq n$)。

具体处理规则：

若 i 为质数，根据性质 1， $\varphi(i) = i - 1$ ；

若 prime_j 整除 i ，则 $\varphi(i \times \text{prime}_j) = \varphi(i) \times \text{prime}_j$ ；

否则， $\varphi(i \times \text{prime}_j) = \varphi(i) \times (\text{prime}_j - 1)$ 。

若只需计算单个 $\varphi(n)$ ，可利用性质 3 在 $O(\sqrt{n})$ 时间内求出，对应模板题中即采用此方法。

欧拉定理

内容：对于两个正整数 a, n ，若 $\gcd(a, n) = 1$ ，则

$$a^{\{\varphi(n)\}} \equiv 1 \pmod{n}$$

费马小定理是欧拉定理的特例：若 $\gcd(a, p) = 1$ 且 p 为质数，则

$$a^{\{p-1\}} \equiv 1 \pmod{p}$$

证明：

设有 $\varphi(n)$ 个数 $x_1, x_2, \dots, x_{\varphi(n)}$ ，满足对于任意 $1 \leq i \leq \varphi(n)$ ： $\gcd(x_i, n) = 1$ 且 $x_i < n$ 。易见这 $\varphi(n)$ 个数模 n 两两不同且余数均与 n 互质。

考虑 $a \cdot x_1, a \cdot x_2, \dots, a \cdot x_{\varphi(n)}$ 也满足同样的两个性质：

两两不同余：反证法。若存在 $i < j$ 使 $a \cdot x_i \equiv a \cdot x_j \pmod{n}$ ，则 $a(x_i - x_j) \equiv 0 \pmod{n}$ 。由于 $\gcd(a, n) = 1$ ，且 $|x_i - x_j| < n$ 且不为 n 的倍数，矛盾。故模 n 两两不同。

余数与 n 互质：由 $\gcd(a, n) = 1$ 及 $\gcd(x_i, n) = 1$ 可得 $\gcd(a \cdot x_i, n) = 1$ 。

因此 $a \cdot x_1, a \cdot x_2, \dots, a \cdot x_{\varphi(n)}$ 的模 n 余数构成与 $x_1, \dots, x_{\varphi(n)}$ 相同的一个排列。于是：

$$(a \cdot x_1)(a \cdot x_2) \dots (a \cdot x_{\varphi(n)}) \equiv x_1 \cdot x_2 \cdot \dots \cdot x_{\varphi(n)} \pmod{n}$$

两边消去与 n 互质的乘积 $x_1 \cdot x_2 \cdot \dots \cdot x_{\varphi(n)}$ ，即得：

$$a^{\{\varphi(n)\}} \equiv 1 \pmod{n}$$

拓展欧拉定理

内容：若 $b \geq m$ 且 $\gcd(a, m) \neq 1$ ，则

$$a^{\{b\}} \equiv a^{\{b \bmod \varphi(m) + \varphi(m)\}} \pmod{m}$$

未包含情况的处理：

若 $b < m$ ，直接使用快速幂计算；

若 $\gcd(a, m) = 1$ ，由欧拉定理有 $a^{\{b\}} \equiv a^{\{b \bmod \varphi(m)\}} \pmod{m}$ 。

证明思路：首先证明对任意质数 p 成立。设 $m = s \times p^k$ 且 $\gcd(s, p) = 1$ 。根据欧拉定理有 $p^{\{\varphi(s)\}} \equiv 1 \pmod{s}$ 。由 $\varphi(s)$ 整除 $\varphi(m)$ 得 $p^{\{\varphi(m)\}} \equiv 1 \pmod{s}$ 。两边同乘 p^k 得 $p^{\{\varphi(m)+k\}} \equiv p^k \pmod{m}$ 。对于 $b \geq k$ ，有 $p^{\{b\}} \equiv p^{\{\varphi(m)+b\}} \pmod{m}$ 。且可证明 $k \leq \varphi(m)$ ，因此当 $b \geq \varphi(m)$ 时结论成立。对质数的幂次方同理，最后由唯一分解定理合并即得一般情况。

代码

```

#include<bits/stdc++.h>
using namespace std;
typedef __int128 LL;

LL Phi(LL n) {
    LL ans = n;
    for (LL i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            ans -= ans / i;
            while (n % i == 0) n /= i;
        }
    }
    if (n > 1) ans -= ans / n;
    return ans;
}

LL QuickPow(LL n, LL p, LL mod) {
    LL base = n, ans = 1;
    while (p) {
        if (p & 1) ans = ans * base % mod;
        p >>= 1;
        base = base * base % mod;
    }
    return ans % mod;
}

LL mod;
bool flag;

LL read() {
    LL x = 0;
    char c = getchar();
    while (c < '0' || c > '9') c = getchar();
    while (c >= '0' && c <= '9') {
        x *= 10; x += (c ^ '0');
        if (x >= mod) flag = true;
        x %= mod; c = getchar();
    }
    return x;
}

LL readI() {
    LL x = 0;

```

```
char c = getchar();
while (c < '0' || c > '9') c = getchar();
while (c >= '0' && c <= '9') {
    x *= 10; x += (c ^ '0');
    c = getchar();
}
return x;
}
```

```
void write(LL x) {
    if (x < 0) putchar('-'), x = -x;
    if (x > 9) write(x / 10);
    putchar(x % 10 + '0');
}
```

```
int main() {
    LL a = readI(), m = readI();
    mod = Phi(m);
    LL b = read();
    if (flag) b += mod;
    write(QuickPow(a, b, m));
    return 0;
}
```